影像處理 Image Processing

大學智慧機器人學程 | 高中 AI 機器人課程 | 台灣自編大學教科書

編者：李世淵(機器人叫獸)|助教：李天宇、李宇晴

第 4 章 幾何轉換

本章學習目標

- 以矩陣表示平移、旋轉、縮放等基本幾何轉換，並說明齊次座標存在的理由。
- 區分剛體、相似、仿射與透視轉換的自由度、不變性質與適用場景。
- 解釋反向映射為何需要內插，比較四種內插法的品質、速度與適用時機。
- 描述針孔相機模型的內部參數與外部參數，辨識桶狀與枕狀鏡頭畸變。
- 以張氏標定法完成相機校正與去畸變，並以重投影誤差驗收成果。
- 運用透視轉換完成文件拉正與機器人導航用的鳥瞰圖。
- 理解幾何轉換在資料增強、視訊穩定、影像拼接與機器人座標系中的角色。

叫獸開講|張正友與一張棋盤格：讓相機校正走進每個人口袋

要讓相機從「拍照的機器」升級成「測量的儀器」，第一步是校正(Calibration)：精確求出它的焦距、光心位置與鏡頭畸變參數。在 1990 年代，這是一件昂貴的苦差事。經典方法需要精密加工的三維標定物——例如兩三個互相垂直、貼滿精確標記點的金屬平面，加工精度直接決定校正精度，一般實驗室根本玩不起。另一派「自我校正」方法不需標定物，數學卻極其艱深且不穩定。相機校正，就這樣卡在「太貴」與「太難」之間。

1998 年 12 月 2 日，微軟研究院的張正友(Zhengyou Zhang)發表了技術報告 MSR-TR-98-71《A Flexible New Technique for Camera Calibration》，給出了第三條路。他的方法簡單到令人難以置信：只需要把一張棋盤格圖案印在紙上、貼到一塊平板上，讓相機從至少兩個不同方位觀察它就行了——而且相機動或棋

盤動都可以，更關鍵的是，你完全不需要知道它是怎麼動的。

技術上，這個方法先利用「平面與其影像之間的單應性」導出對內部參數的約束，以封閉解求出初始估計，再以最大概似準則做非線性最佳化，同時把徑向畸變一併建模。論文於 1999 年在頂級會議 ICCV 發表，並於 2000 年正式刊登在頂級期刊 IEEE TPAMI 上。張正友在論文中寫下的一句話，精準地概括了它的意義：相較於需要兩三個正交平面等昂貴設備的傳統方法，這個技術簡單而靈活，讓三維電腦視覺從實驗室環境往真實世界前進了一步。

「一張紙取代精密機械」的威力是巨大的。任何人花五分鐘列印一張棋盤格，就能完成過去要專業設備才能做的事。這個方法後來成為 OpenCV 內建的標準校正流程(cv2.calibrateCamera 的核心正是張氏標定法)，也是全球手機、無人機、機器人出廠校正管線的基礎，論文引用數以萬計，是電腦視覺史上影響力最大的工作之一。張正友後來持續在工業界深耕電腦視覺與機器人研究——而你今天在 Lab 4-2 印出的那張棋盤格，正是這段歷史的延續。

叫獸想從這裡點出一個貫穿全書的觀察。回想第 2 章 OpenCV 的開源、第 13 章 ORB 對 SIFT 專利的回應、第 17 章五美元的 ESP32——這些改變領域的突破，往往不是「做出更強大的東西」，而是「大幅降低了做這件事的門檻」。把一項只有少數人玩得起的技術，變成每個人都能用的工具，其影響力常常遠超過在精度上多推進幾個百分點。這是工程創新中最容易被低估、卻最有力量的一種。

思考題：張氏標定法要求棋盤格「至少兩個不同方位」，但實務上大家都拍 10 到 20 張。想一想：多拍的那些照片，是在對抗什麼?(提示：雜訊、角點定位誤差，以及涵蓋畫面各個角落的畸變。張正友原始論文的實驗也顯示，估計結果的不確定度會隨影像張數增加而下降)

4.1 基本幾何轉換與齊次座標

4.1.1 幾何轉換改變什麼

先釐清一件事：幾何轉換改變的是像素的「位置」，而非像素的「數值」。第 5 章的灰階轉換是「位置不變、數值改變」，本章的幾何轉換則恰好相反——把影像中的內容搬到別的地方去，而每個點的亮度或顏色原則上跟著它一起搬。這個對照有助於你建立整體樣貌：影像處理的操作，不是在動數值，就是在動位置，或兩者兼有。

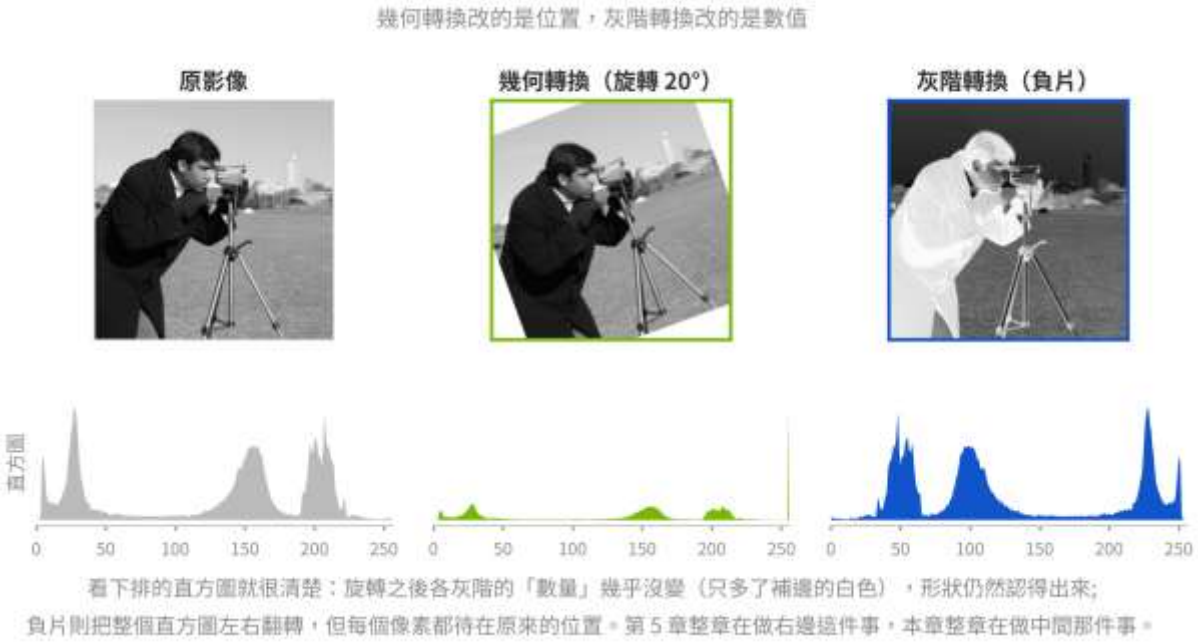


圖 4-1　幾何轉換改位置，灰階轉換改數值

4.1.2 三種基本轉換

三種最基本的轉換是：平移把每個點 (x, y) 移到 (x + tx, y + ty);縮放把座標乘上倍率，得到 (sx·x, sy·y);旋轉將點繞原點轉 θ 角，新座標為 x' = x·cosθ - y·sinθ,y' = x·sinθ + y·cosθ。

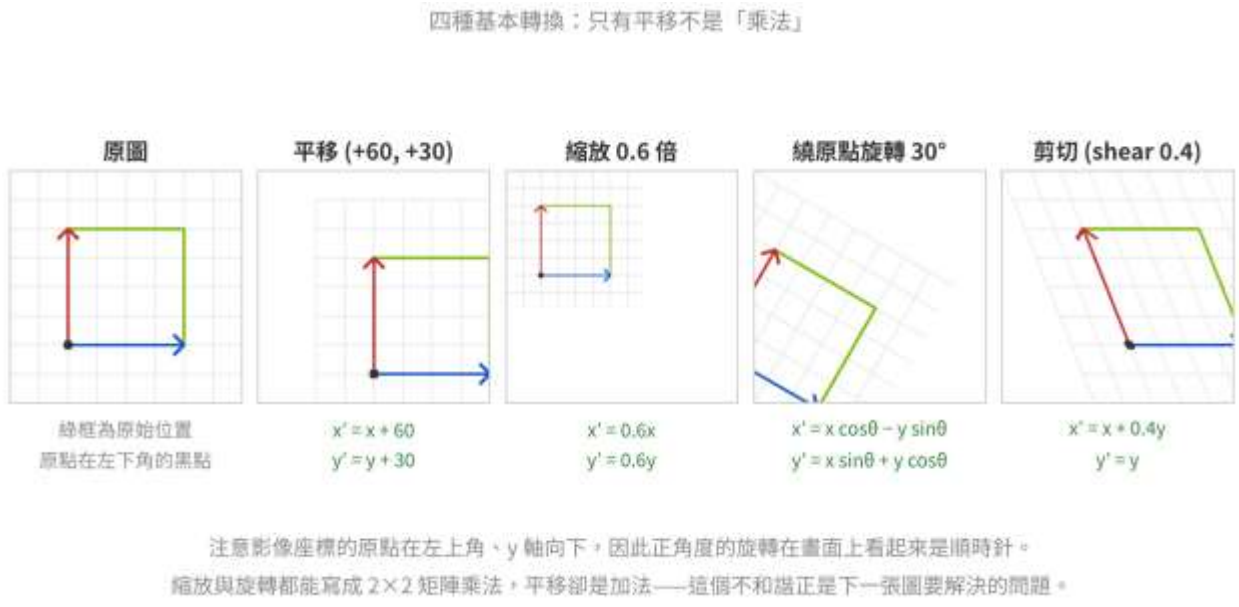


圖 4-2　四種基本轉換：只有平移不是乘法

縮放與旋轉都能寫成 2×2 矩陣乘法，唯獨平移是「加法」，格格不入。這個小小的不和諧看似無關痛癢，實際上很麻煩：當你要串接多個轉換時，乘法與加法混在一起，無法合成為單一運算。這正是齊次座標登場的理由。

4.1.3 齊次座標：讓所有轉換說同一種語言

齊次座標的做法很簡單：替每個點多加一維，把 (x, y) 寫成 (x, y, 1)。如此一來，平移也能表示成 3×3 矩陣乘法，三種轉換全部統一成「矩陣乘以向量」的形式。

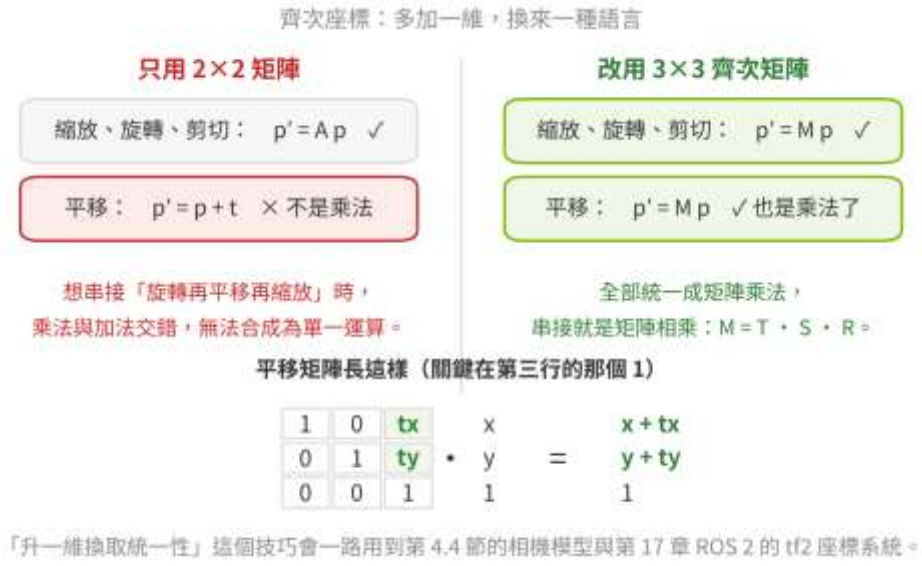


圖 4-3　齊次座標：多加一維，把平移也變成乘法

統一的好處立刻顯現。第一，多個轉換的串接，等於把矩陣依序相乘成一個矩陣——「先旋轉 30 度、再放大 1.5 倍、再平移」三個動作，可以合成一個 3×3 矩陣一次做完。第二，這不只是寫法漂亮：一次完成比分三次做更快，而且更重要的是避免了多次取樣造成的品質劣化(每做一次轉換就要內插一次，而每次內插都會損失一點細節，第 4.3 節詳述)。

請特別留意矩陣相乘的順序：轉換是從右往左作用的。若寫成 M = T · S · R，代表先旋轉、再縮放、最後平移。順序不可交換——先旋轉再平移，與先平移再旋轉，結果完全不同(想像繞原點旋轉的圓心位置)。這是初學者最常見的錯誤之一。

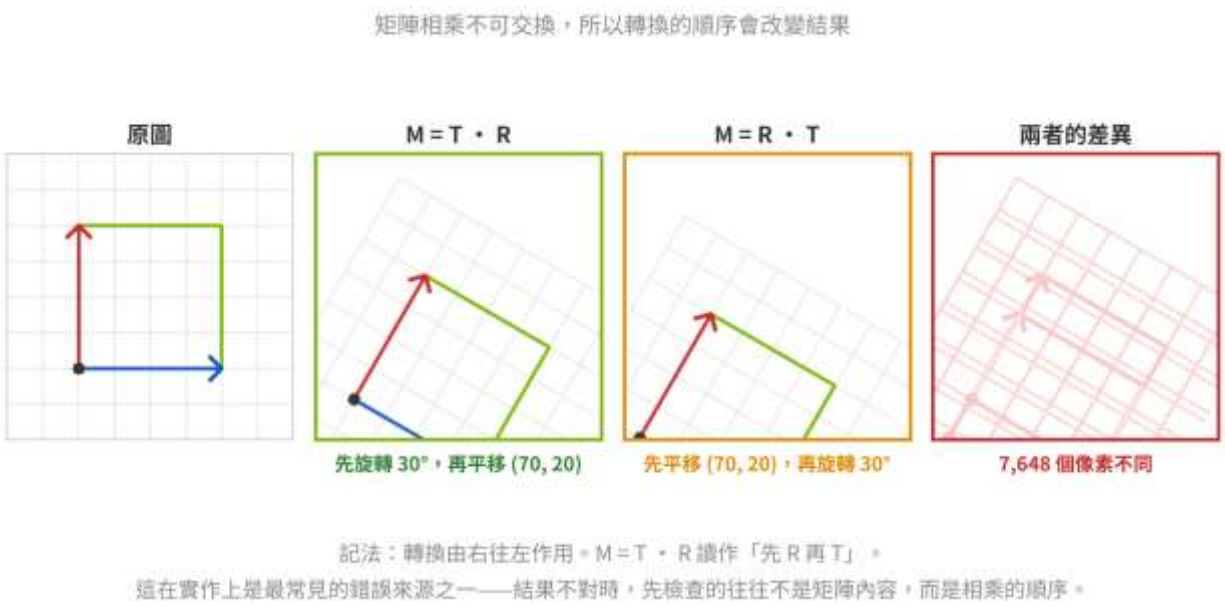


圖 4-4　轉換順序不可交換

「升一維換取統一性」這個技巧，其影響遠超出本章。第 4.4 節的相機模型、第 17 章 ROS 2 的座標轉換系統(tf2)，背後都是齊次座標。一台機器人身上有相機座標系、機械臂座標系、底盤座標系、世界座標系，而 tf2 就是以齊次座標矩陣管理這些座標系之間的關係——當你在第 17 章看到 tf2 時，請記得它的根就在這裡。



圖 4-5　分三次做與合成一次做的品質差距

程式 4-1 平移、旋轉、縮放與串接

```
import cv2
import numpy as np

img = cv2.imread("robot.jpg")
h, w = img.shape[:2]

# --- 平移:x 方向 +100,y 方向 +50 ---
M_t = np.float32([[1, 0, 100],
                  [0, 1, 50]])          # 2x3 仿射矩陣
shifted = cv2.warpAffine(img, M_t, (w, h))

# --- 旋轉:繞影像中心轉 30 度,順便縮放 0.8 倍 ---
M_r = cv2.getRotationMatrix2D(center=(w/2, h/2), angle=30, scale=0.8)
rotated = cv2.warpAffine(img, M_r, (w, h))

# --- 縮放:resize 是最常用的幾何轉換 ---
half = cv2.resize(img, None, fx=0.5, fy=0.5,
                  interpolation=cv2.INTER_AREA)   # 縮小用 AREA(第 3 章)

# --- 串接:合成單一矩陣,只取樣一次 ---
R = np.vstack([M_r, [0, 0, 1]])          # 補成 3x3 齊次矩陣
T = np.float32([[1, 0, 100], [0, 1, 50], [0, 0, 1]])
M = T @ R                                 # 先旋轉、再平移(從右往左)
combined = cv2.warpAffine(img, M[:2], (w, h))
# 比較:分兩次做 vs. 合成一次做,後者更快且細節損失更少
```

4.1.4 推導：旋轉矩陣從哪裡來，以及繞任意點旋轉

旋轉矩陣的四個元素常被當成公式背起來，但它其實可以在三十秒內推出來，而

且推完之後「繞任意點旋轉」也就順便解決了。

先推旋轉矩陣。任何線性轉換，都完全由「它把兩個基底向量送到哪裡」決定——因為任意向量都是基底的線性組合，而線性轉換保持線性組合。所以只要問兩個問題：把 x 軸的單位向量 (1, 0) 繞原點轉 θ 角，它會到哪裡?答案直接由三角函數定義：(cos θ, sin θ)。把 y 軸的單位向量 (0, 1) 轉 θ 角呢?它會到 (−sin θ, cos θ)。而矩陣的第一行就是「x 軸被送到哪」、第二行就是「y 軸被送到哪」，於是旋轉矩陣必然是 [[cos θ, −sin θ], [sin θ, cos θ]]。這個「看基底去了哪裡」的技巧對所有線性轉換都適用，縮放、剪切都能這樣一眼寫出矩陣。

提醒一個影像特有的陷阱：上面的推導假設 y 軸向上，但影像座標的 y 軸向下（第 2.2.1 節）。因此同一個矩陣作用在影像上時，正的 θ 在畫面上看起來是順時針旋轉。這不是矩陣錯了，是座標系的方向不同——圖 4-2 標示的就是影像座標下的實際效果。

接著是繞任意點旋轉。旋轉矩陣是繞原點轉的，而影像的原點在左上角，所以直接套用會讓影像繞著左上角甩出去。要繞中心 (cx, cy) 轉，做法是三步：先把中心平移到原點、在原點做旋轉、再平移回去。寫成矩陣就是 M = T(cx, cy) · R(θ) · T(−cx, −cy)，記得由右往左讀。

這個「搬過去、做事、搬回來」的三明治結構，在數學上稱為共軛（conjugation），是幾何轉換中最常用的一招。繞任意點縮放、沿任意一條直線鏡射、繞任意軸旋轉（三維），用的都是同一個模式：先用一個轉換把問題搬到「簡單的位置」，在那裡做事，再用反轉換搬回來。OpenCV 的 cv2.getRotationMatrix2D(center, angle, scale) 內部做的正是這件事，它回傳的是合成後的 2×3 矩陣——這也解釋了為什麼它需要 center 這個參數。

最後補一個在實作上救命的檢查法。當結果不如預期時，不要盯著影像猜，直接拿幾個已知的點代進矩陣算一遍：影像的四個角分別被送到哪裡?中心點有沒有留在原

處?三十秒的手算往往比三十分鐘的目視除錯更有效，而且它會明確告訴你問題出在矩陣內容還是相乘順序。

4.2 轉換家族：從剛體到透視

幾何轉換構成一個層層包含的家族，自由度愈高、能表達的變形愈多、但保持的性質愈少。理解這個家族譜，你才知道面對一個問題該選哪一種。

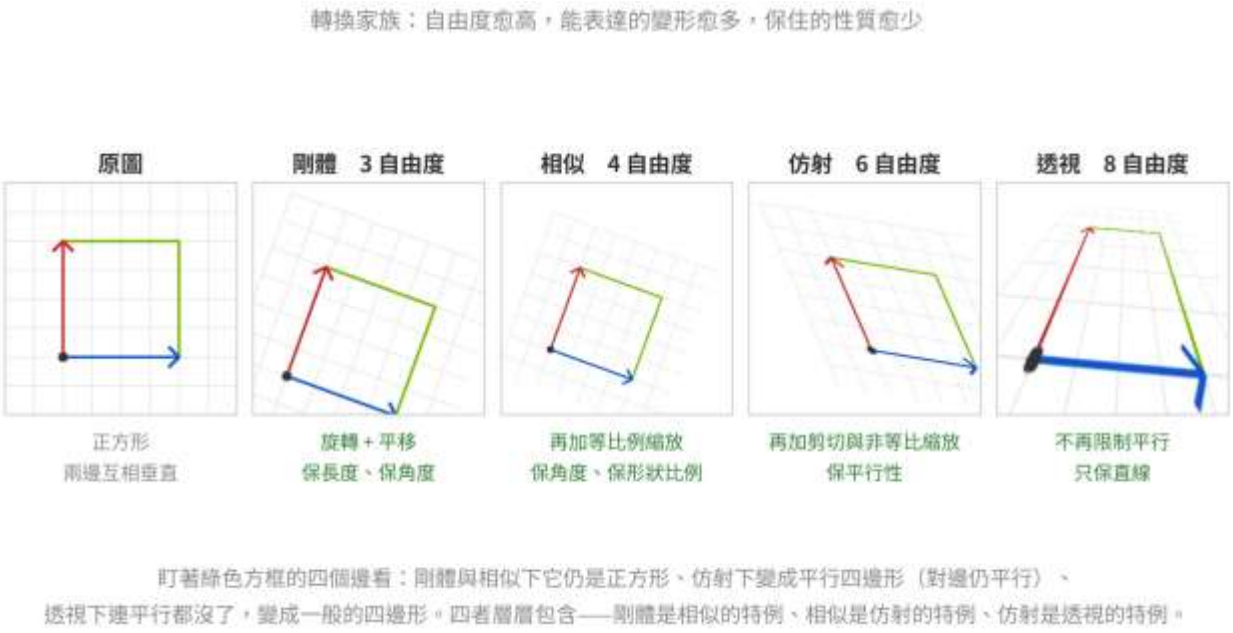


圖 4-6　轉換家族：自由度愈高，保住的性質愈少

1. 剛體轉換(Rigid / Euclidean)：只有旋轉與平移，3 個自由度。保持長度與角度——物體不變形，只是換了位置與朝向。機器人的移動、物件的擺放姿態屬於此類。
2. 相似轉換(Similarity)：剛體再加上等比例縮放，4 個自由度。保持角度與形狀比例，但大小可變。照片的等比例縮放、無人機拉高後看到的地面屬於此類。
3. 仿射轉換(Affine)：再加上剪切與非等比例縮放，2×3 矩陣、6 個自由度、需 3 組對應點。保持平行性——轉換前平行的線，轉換後仍平行(長度與角度可變)。

4. 透視轉換(Perspective / Homography):3×3 矩陣、8 個自由度、需 4 組對應點。不保平行但保直線——平行線可以相交(這正是近大遠小的視覺效果)，而直線轉換後仍是直線。

4.2.1 仿射轉換：平行線的守護者

仿射轉換是平移、旋轉、縮放、剪切的任意組合。因為 6 個自由度對應 3 個點的對應關係，cv2.getAffineTransform() 只需要「轉換前後各三個點」就能求出矩陣。



圖 4-7　自由度決定需要幾組對應點

典型應用有二。其一是影像對齊：把兩張有輕微位移旋轉的影像疊合(第 12 章的視訊穩定即屬此類)。其二是資料增強——第 14、15、16 章訓練神經網路時的隨機旋轉、平移、翻轉、剪切，本質上全是仿射轉換。這個連結值得記住：當你在第 15 章看到 RandomRotation 這類增強參數時，它呼叫的就是本章的技術。

4.2.2 透視轉換：拍歪的世界拉回正面

用相機斜拍一張 A4 紙，紙的影像不再是矩形而是梯形——原本平行的紙邊，在影像中相交了。要描述這種「近大遠小」的投影，就需要透視轉換，又稱單應性(Homography)。它需要 4 組對應點，由 cv2.getPerspectiveTransform() 求解。



圖 4-8　透視轉換：把拍歪的文件拉回正面

透視轉換是本章最實用的工具，它的應用橫跨全書：文件掃描 App 的「拉正」功能、第 13 章車牌影像的正面化、機器人導航的鳥瞰圖、第 13 章影像拼接求兩張照片間的關係、以及第 13.4.3 節 RANSAC 求出的正是這個矩陣。

一個重要的觀念補充：單應性成立有其前提——要嘛拍攝的是一個平面(如紙張、地面、牆面)，要嘛相機只做純旋轉而不平移。若場景有明顯的深度變化而相機又平移了，單一的單應性就無法描述兩張影像的關係(這也是為什麼手持平移拍攝的照片較難拼接，而原地旋轉拍攝的全景圖很容易)。理解這個限制，能幫你判斷什麼時候該用透視轉換、什麼時候需要更複雜的三維方法。

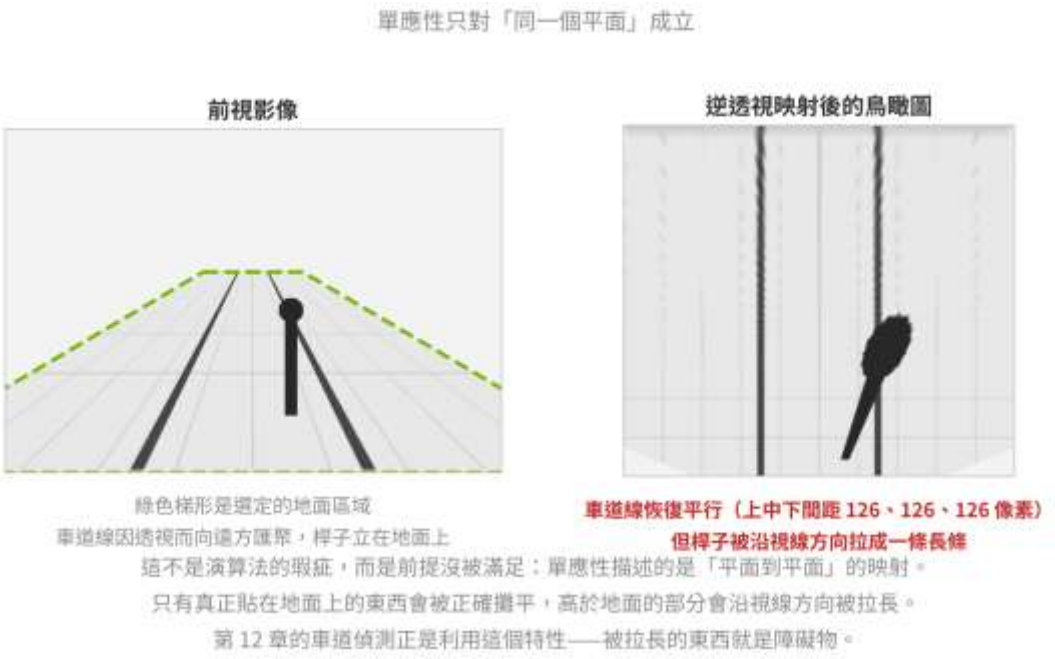


圖 4-9　單應性只對同一個平面成立

轉換	自由度	對應點數	保持的性質	典型應用
剛體	3	2	長度、角度	物件姿態、機器人位姿
相似	4	2	角度、形狀比例	等比縮放、尺度變化
仿射	6	3	平行性、面積比	影像對齊、資料增強
透視	8	4	直線性、交比	文件拉正、鳥瞰圖、拼接

表 4-1 幾何轉換家族比較

程式 4-2 透視轉換：文件拉正

```
import cv2
import numpy as np

img = cv2.imread("tilted_doc.jpg")

# 斜拍文件的四個角(左上、右上、右下、左下)
src = np.float32([[168, 92], [583, 128], [612, 707], [95, 664]])

# 自動計算合理的長寬比,避免文件被不自然拉伸
def dist(a, b): return np.linalg.norm(a - b)
W = int((dist(src[0], src[1]) + dist(src[3], src[2])) / 2)
H = int((dist(src[0], src[3]) + dist(src[1], src[2])) / 2)

dst = np.float32([[0, 0], [W, 0], [W, H], [0, H]])
```

```
M = cv2.getPerspectiveTransform(src, dst)   # 3x3 透視矩陣
flat = cv2.warpPerspective(img, M, (W, H))
cv2.imwrite("document_flat.jpg", flat)      # 文件掃描 App 的核心

# 反向:也可把矩陣反轉,把正面圖貼回原始視角(擴增實境的原理)
M_inv = np.linalg.inv(M)
```

4.2.3 推導：單應性為什麼是八個自由度，四個點又是怎麼解出來的

4.2 節說透視轉換是「3×3 矩陣、8 個自由度、需 4 組對應點」。這三個數字彼此相關，而且都能推出來。

先看自由度。3×3 矩陣有九個元素，看起來是九個自由度，但在齊次座標下，把整個矩陣乘上任一個非零常數 c，轉換後的點 (cx′, cy′, cw′) 與 (x′, y′, w′) 在除以最後一維之後會得到完全相同的結果。換句話說，整體尺度是看不見的，九個元素中只有八個是真正獨立的。實作上通常把 h33 固定為 1 來消去這個多餘的自由度，或要求矩陣的範數為 1。

再看為什麼是四個點。每一組對應點 (x, y) → (x′, y′) 提供兩個方程式。把齊次座標的轉換展開：x′ = (h11·x + h12·y + h13) / (h31·x + h32·y + h33)，y′ = (h21·x + h22·y + h23) / (h31·x + h32·y + h33)。注意分母——這個除法正是透視與仿射的根本差異，也是「近大遠小」的數學來源：分母隨位置而變，所以遠處的東西被壓得比較小。把分母乘過去，兩式就變成對 h 的線性方程。八個未知數需要八個方程式，而每組點給兩個，故需 4 組——而且這四個點不能有任何三點共線，否則方程式之間會線性相依，解不出來。實作上這代表：選文件的四個角時要選真正的四個角，不要選同一條邊上的四個點。

解法的標準流程稱為直接線性轉換（DLT）：把八個方程式整理成 A·h = 0 的形式，其中 h 是把九個元素排成的向量、A 是 8×9 的矩陣，再以奇異值分解（SVD）求 A 的最小奇異值對應的奇異向量。這個解在最小平方的意義下最接近零空間，正好對應「尺度不計」的性質。OpenCV 的 cv2.getPerspectiveTransform 做的就是四點的精確解，cv2.findHomography 則接受更多點並以最小平方或 RANSAC 求解。

最後回到前提。這整套推導從頭到尾假設了「存在一個 3×3 矩陣能描述兩張影像之間的對應」，而這件事只在兩種情況下成立（4.2.2 節與圖 4-9）：場景是一個平面，或相機只做純旋轉。若前提不成立，上面的方程組仍然解得出一個答案——這正是危險的地方。演算法不會報錯，它會給你一個「在最小平方意義下最好」的矩陣，而那個矩陣誰也對不齊。遇到這種情形時，要換的不是參數而是模型：第 12 章的光流或完整的三維重建才描述得了「不同深度有不同位移」。

4.3 影像內插：轉換後的像素從哪裡來

4.3.1 為什麼要反向映射

直覺上，執行幾何轉換似乎應該「把每個輸入像素搬到它該去的位置」——這稱為正向映射。但這樣做會產生兩個嚴重問題：某些輸出位置沒有任何輸入像素落上去，形成破洞;而某些位置則有多個輸入像素重疊，不知該用哪一個。

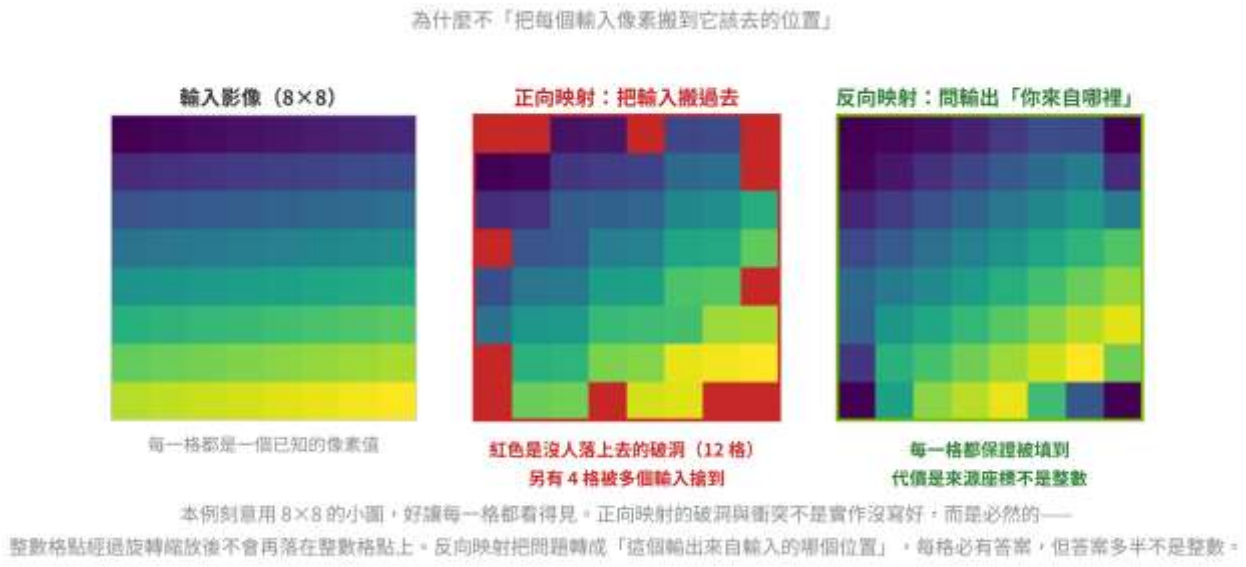


圖 4-10　正向映射的破洞與衝突，以及反向映射的解法

因此 OpenCV 實際採用反向映射(Backward Mapping)：對輸出影像的每一個像素位置，用轉換矩陣的反矩陣反算「它來自輸入影像的哪裡」。這樣每個輸出像素都保證有值、且只有一個來源，沒有破洞也沒有衝突。

但新問題來了：反算出的來源座標幾乎都不是整數——像 (217.3, 154.8) 這樣的

位置根本沒有現成的像素值。從鄰近的整數像素「估」出這個值，就是內插(Interpolation)。這也解釋了為什麼幾何轉換必然伴隨品質損失：每一次轉換都要內插一次，而內插本質上是猜測。

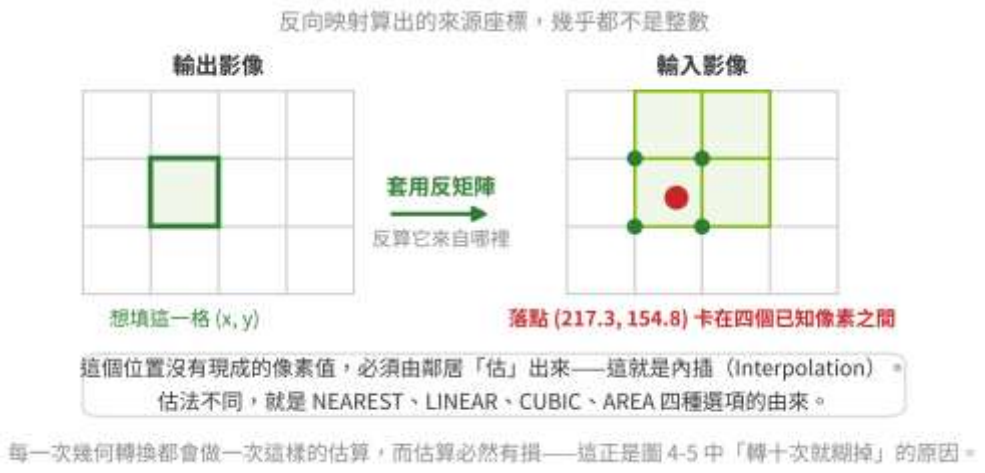


圖 4-11　反算出的來源座標幾乎都不是整數

4.3.2 四種內插法

5. 最近鄰內插(INTER_NEAREST)：直接取最近的那個像素值。最快，但放大後鋸齒與方塊感明顯。它唯一但關鍵的優點是不會產生「新的數值」——因此處理標籤圖、分割遮罩(第 9、16 章)時必須用它。若對一張只有 0 與 1 的遮罩用雙線性內插，會產生 0.5 這種無意義的中間值。

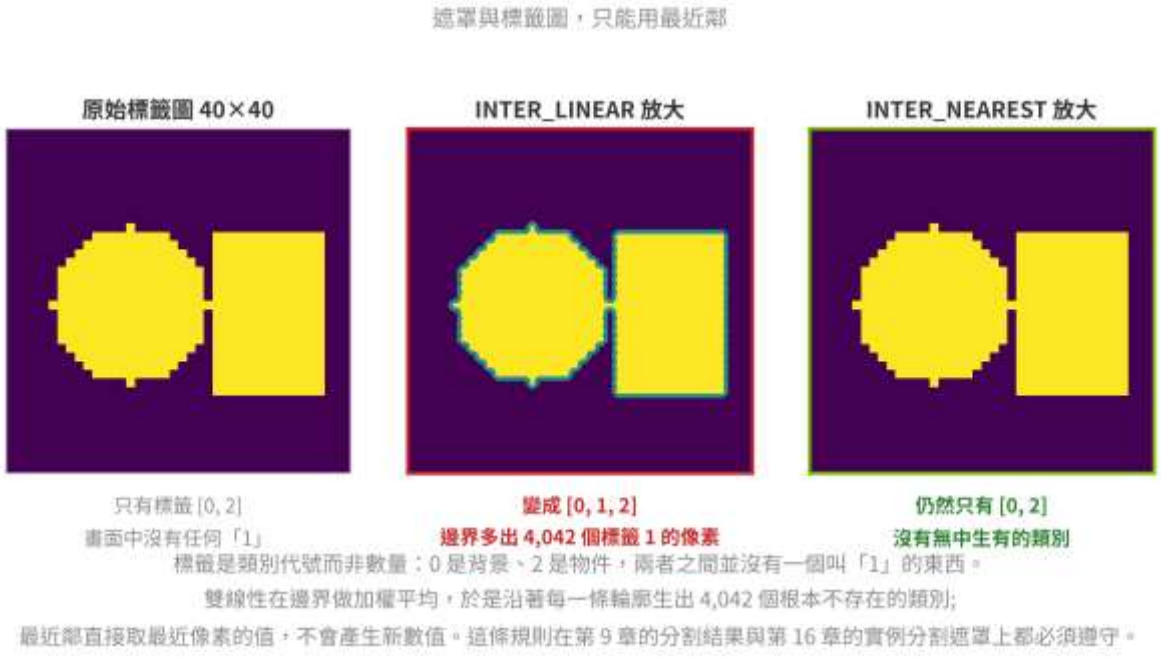


圖 4-13　遮罩與標籤圖只能用最近鄰

6. 雙線性內插(INTER_LINEAR,OpenCV 預設)：取周圍 2×2 共 4 個像素，依距離加權平均。速度與品質的甜蜜點，絕大多數場合的首選。
7. 雙三次內插(INTER_CUBIC)：取周圍 4×4 共 16 個像素，以三次多項式加權。邊緣更平滑銳利，代價是約數倍的運算量，適合離線的高品質放大。

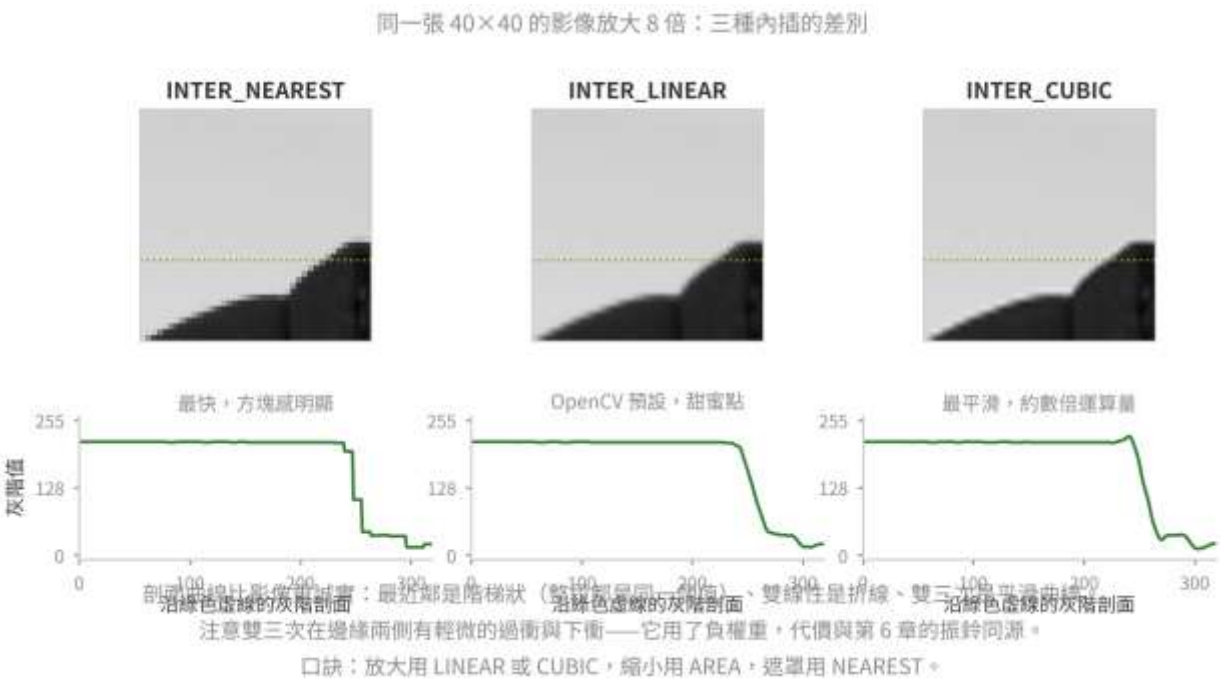


圖 4-12　三種內插法放大八倍：影像與灰階剖面

8. 區域內插(INTER_AREA)：以區域平均進行降取樣。它是縮小影像的建議選項——原因在第 3 章已經說明：縮小就是降取樣，不先做低通就會產生混疊，而 INTER_AREA 的區域平均正是內建的低通濾波。

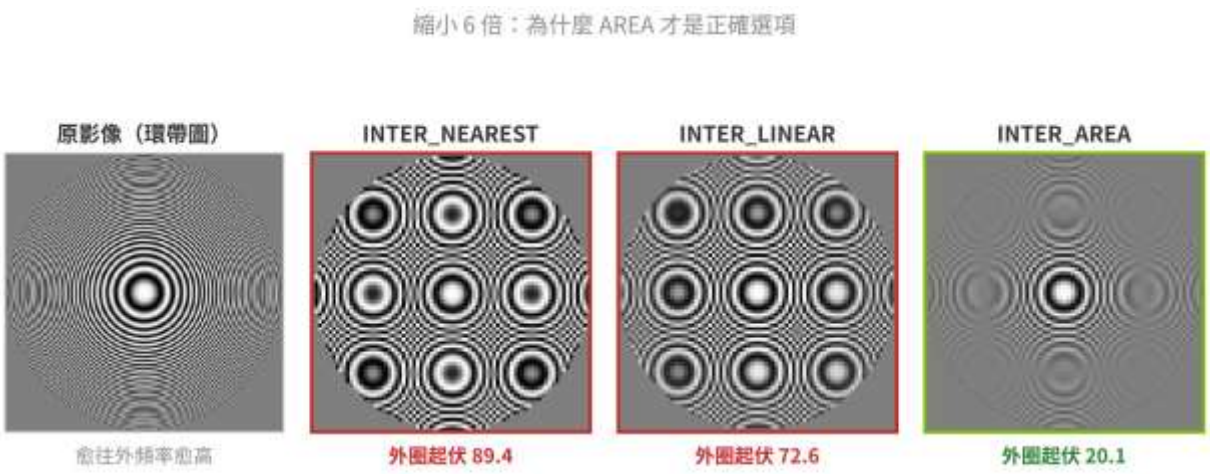


圖 4-14　縮小影像為什麼要用 INTER_AREA

口訣：放大用 LINEAR 或 CUBIC，縮小用 AREA，遮罩用 NEAREST。這個口訣背後串起了第 3 章(取樣定理與混疊)、第 5 章(平滑濾波)、第 9 章(分割遮罩)三章的觀念。

程式 4-3 四種內插法與多次轉換的代價

```
import cv2, numpy as np

img = cv2.imread("small_photo.jpg")

# --- 放大:比較三種內插的品質 ---
for name, flag in [("nearest", cv2.INTER_NEAREST),
                   ("linear",  cv2.INTER_LINEAR),
                   ("cubic",   cv2.INTER_CUBIC)]:
    big = cv2.resize(img, None, fx=4, fy=4, interpolation=flag)
    cv2.imwrite(f"up_{name}.png", big)
# 放大 4 倍後檢視:nearest 滿是方塊,cubic 邊緣最平滑

# --- 遮罩必須用 NEAREST,否則會產生無意義的中間值 ---
mask = np.zeros((100, 100), np.uint8)
mask[30:70, 30:70] = 1                          # 標籤圖:只有 0 與 1

bad  = cv2.resize(mask, (400, 400), interpolation=cv2.INTER_LINEAR)
good = cv2.resize(mask, (400, 400), interpolation=cv2.INTER_NEAREST)
print("雙線性後的唯一值:", np.unique(bad))    # 出現 0 與 1 以外的值!
print("最近鄰後的唯一值:", np.unique(good))   # 仍只有 0 與 1

# --- 多次轉換 vs. 合成一次:品質差異 ---
a = img.copy()
for _ in range(10):                             # 連續旋轉 10 次,每次 36 度
    M = cv2.getRotationMatrix2D((img.shape[1]/2, img.shape[0]/2), 36, 1)
    a = cv2.warpAffine(a, M, img.shape[1::-1])
```

```
print("轉一圈後與原圖的 PSNR:", cv2.PSNR(img, a))
# 理論上轉回原位,實際上因十次內插而明顯劣化 → 應合成矩陣一次做
```

4.3.3 推導：雙線性內插的公式，與它為什麼會讓影像變糊

雙線性內插是本書使用最頻繁的內插法，它的公式只有一行，但「它為什麼會讓影像變糊」這個問題要到第 6 章才真正說得清楚。這裡把兩件事一次講完。

先看公式。設反算出的來源座標為 (x + a, y + b)，其中 x、y 是整數、a 與 b 是 0 到 1 之間的小數部分。周圍四個已知像素是 I(x, y)、I(x+1, y)、I(x, y+1)、I(x+1, y+1)。做法是先在水平方向做兩次一維線性內插：上列得到 (1−a)·I(x, y) + a·I(x+1, y)，下列得到 (1−a)·I(x, y+1) + a·I(x+1, y+1);再把這兩個結果在垂直方向依 b 內插一次。展開後四個像素的權重分別是 (1−a)(1−b)、a(1−b)、(1−a)b、ab，四者相加恰為 1——這是檢查算式有沒有寫錯的最快方法。

順序可以顛倒：先垂直再水平，結果完全相同。這個性質稱為可分離性，與第 5.3.3 節的可分離濾波同源，也是雙線性內插比雙三次快得多的原因——二維的問題被拆成兩次一維運算。

接著是「為什麼會糊」。關鍵在於換個角度看這件事：雙線性內插其實是把輸入影像與一個三角形的權重函數做卷積，再在新的位置取樣。而卷積就是濾波（第 5 章）——這個三角形權重是一個低通濾波器，它必然衰減高頻成分。第 6 章會給出定量的答案：三角形函數的頻率響應是 sinc 的平方，在 Nyquist 頻率處的增益已經掉到相當低，這就是為什麼每做一次內插，細節就少一分。圖 4-5 實測的「分三次做比合成一次做少了兩成銳利度」，量的正是這個效應。

這個觀點也一併解釋了另外兩件事。其一，雙三次內插為什麼比較銳利卻會過衝：它的權重函數帶有負的部分，等於一個輕微的銳化濾波器，代價就是在強邊緣兩側產生過衝與下衝（圖 4-12 的剖面曲線）——與第 6.3.2 節的振鈴效應是同一個現象的不同程度。其二，最近鄰為什麼不糊卻有鋸齒：它的權重函數是一個方形脈衝，在空間

域完全不平均（所以不糊），但方形在頻率域的旁瓣極多（第 6 章），表現出來就是階梯狀的鋸齒。

結論回到實務：既然每次內插都要付出高頻的代價，最好的策略就是少做幾次。把所有轉換合成單一矩陣、只取樣一次，不只是為了省時間，更是為了省細節。

4.4 相機模型、鏡頭畸變與張氏標定

4.4.1 針孔相機模型

針孔相機模型把成像抽象成一句話：三維空間中的點沿直線投影穿過光心，落在成像平面上。以齊次座標表示，整個投影過程可寫成「內參矩陣 K 乘以外參 [R|t] 乘以世界座標點」——注意這裡再次用到了 4.1.3 節的齊次座標，它讓三維到二維的投影也能寫成單一的矩陣乘法。



圖 4-15　針孔相機模型：三維世界如何變成二維影像

參數分成兩組，區分它們是理解校正的關鍵：

- 內部參數(Intrinsics)：屬於相機本身，包括焦距 fx、fy(以像素為單位)與主點 cx、cy(光軸與成像面的交點，理想上在影像中心)。只要相機與鏡頭組合不變，這組參數就固定，與相機放在哪裡無關。



圖 4-16　內部參數的幾何意義：四個數字各自改變什麼

- 外部參數(Extrinsics)：描述相機在世界中的位置與朝向，包括旋轉矩陣 R 與平移向量 t。相機一移動，這組參數就改變。

這個區分有實務意義：換一顆鏡頭或調整變焦，必須重新校正內參;而把相機搬到另一個腳架上，內參不變、只有外參改變。在機器人系統中，內參通常在出廠時校正一次並存檔，外參則由第 17 章的 tf2 系統即時追蹤。

4.4.2 鏡頭畸變

真實鏡頭不是理想的針孔，光線經過鏡片組會產生畸變(Distortion)。徑向畸變沿半徑方向作用，以係數 k1、k2、k3 建模：係數為負時影像向外鼓、直線變成外凸的弧線，稱為桶狀畸變(廣角鏡、GoPro、ESP32-CAM 的魚眼感都是它);係數為正時向內縮，稱為枕狀畸變，常見於望遠端。切向畸變(p1、p2)則來自鏡片與感光元件未能完全平行。

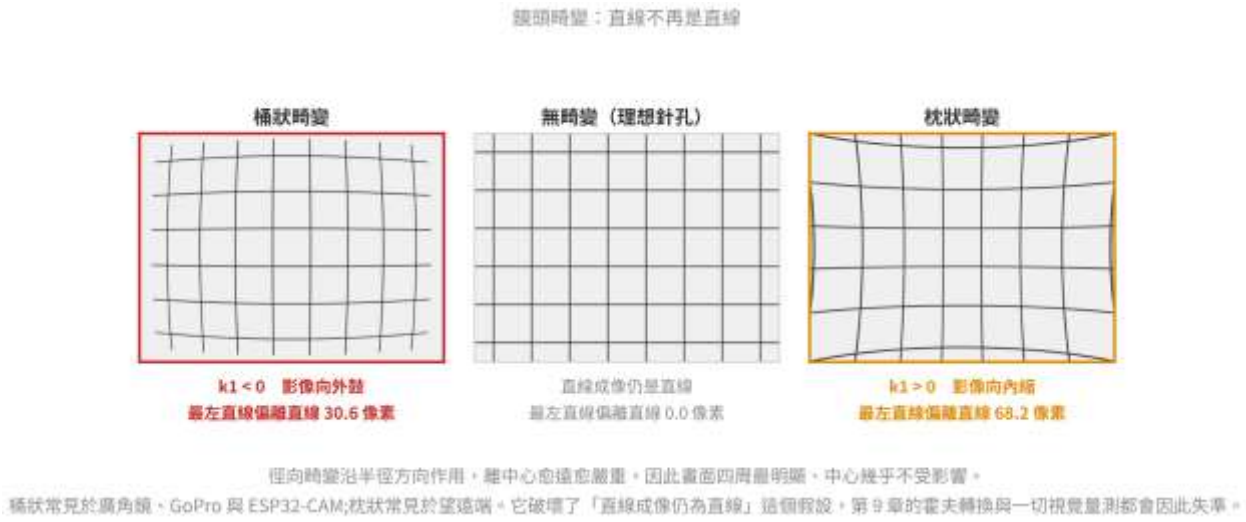


圖 4-17　桶狀與枕狀畸變：直線不再是直線

畸變對機器人與量測應用是致命的，因為它破壞了「直線成像仍是直線」這個假設：第 9 章的霍夫轉換車道偵測會把彎曲的車道線判為多段短線、第 13 章的特徵匹配位置會系統性偏移、第 17 章以像素換算實際距離的計算會失準——而且愈靠近畫面邊緣誤差愈大。因此本書的原則是：先校正，再談視覺量測。

4.4.3 張氏標定法實作

依叫獸開講的原理，OpenCV 的標定流程分四步：拍攝棋盤格多張、偵測角點、求解參數、去畸變。

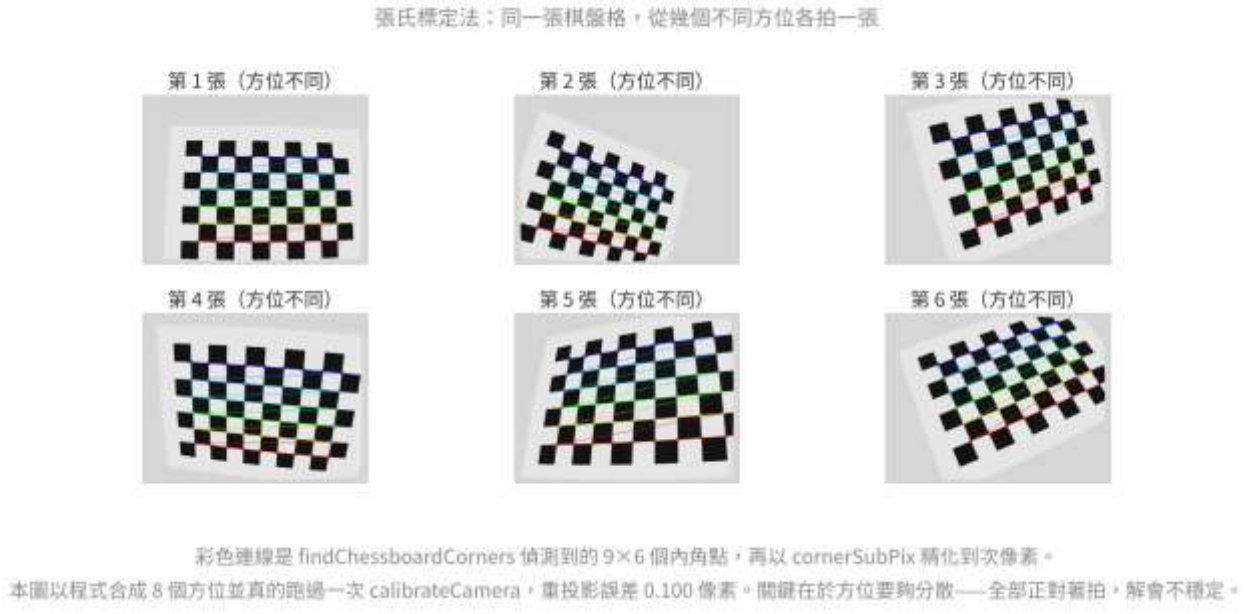


圖 4-18　張氏標定法：從多個方位拍同一張棋盤格

程式 4-4 張氏標定法完整流程

```
import cv2
import numpy as np
import glob

CB = (9, 6)   # 棋盤格「內角點」數:10x7 格的板子內角點為 9x6

# 棋盤格角點的世界座標(z=0 平面,單位:格)
objp = np.zeros((CB[0]*CB[1], 3), np.float32)
objp[:, :2] = np.mgrid[0:CB[0], 0:CB[1]].T.reshape(-1, 2)

objpoints, imgpoints = [], []
for fname in glob.glob("calib/*.jpg"):      # 10~20 張不同方位
    gray = cv2.imread(fname, cv2.IMREAD_GRAYSCALE)
    ok, corners = cv2.findChessboardCorners(gray, CB, None)
    if ok:
        # 次像素精化:把角點定位精度提升到小數點以下
        corners = cv2.cornerSubPix(gray, corners, (11, 11), (-1, -1),
            (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 30, 0.001))
        objpoints.append(objp)
        imgpoints.append(corners)

ret, K, dist, rvecs, tvecs = cv2.calibrateCamera(
    objpoints, imgpoints, gray.shape[::-1], None, None)

print("重投影誤差(愈小愈好,<0.5 佳):", ret)
print("內參矩陣 K:\n", K)          # fx, fy, cx, cy
print("畸變係數:", dist.ravel())    # k1 k2 p1 p2 k3

# 存檔:整學期所有實驗共用(第 12、17 章都會用到)
np.savez("camera_params.npz", K=K, dist=dist)

# 去畸變:彎掉的直線拉回來
img = cv2.imread("calib/test.jpg")
undist = cv2.undistort(img, K, dist)
cv2.imwrite("undistorted.jpg", undist)
```

驗收標準是重投影誤差：把求得的參數反過來把世界點投影回影像，與實際偵測到的角點位置比較，取平均距離。一般以低於 0.5 像素為佳。若誤差偏大，常見原因有三：棋盤格不平整(貼在軟紙板上會微微彎曲)、照片模糊或動態模糊、方位變化不足(所有照片都從相似角度拍)。這正好回應了叫獸開講的思考題——多拍不同方位的照片，就是在對抗這些誤差來源。

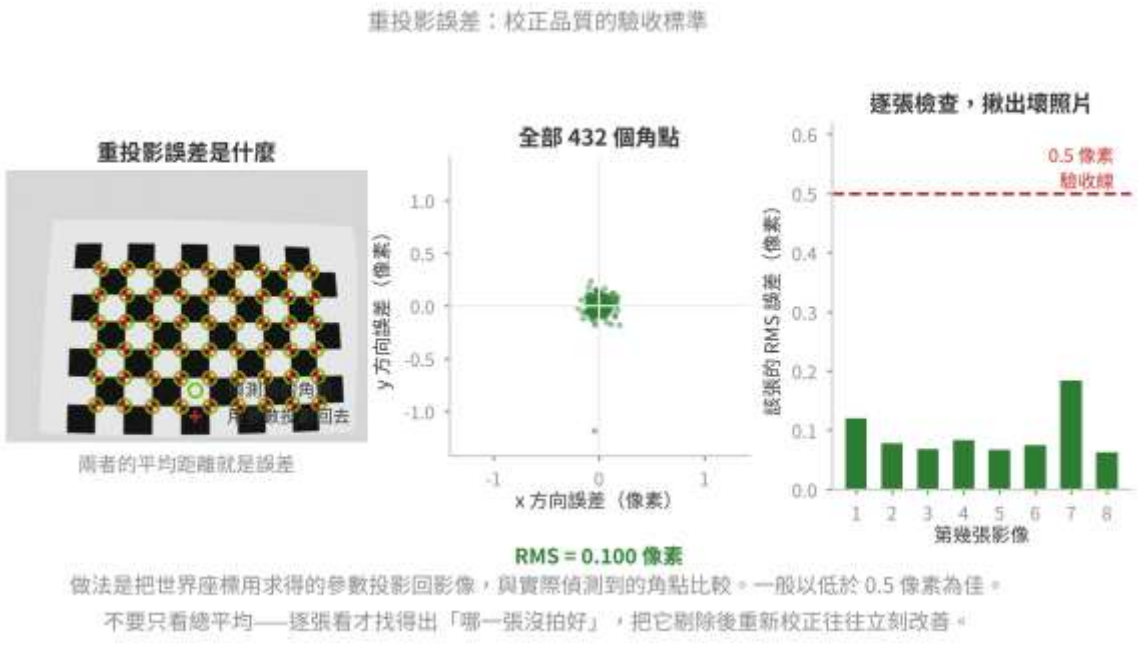


圖 4-20　重投影誤差：校正品質的驗收標準

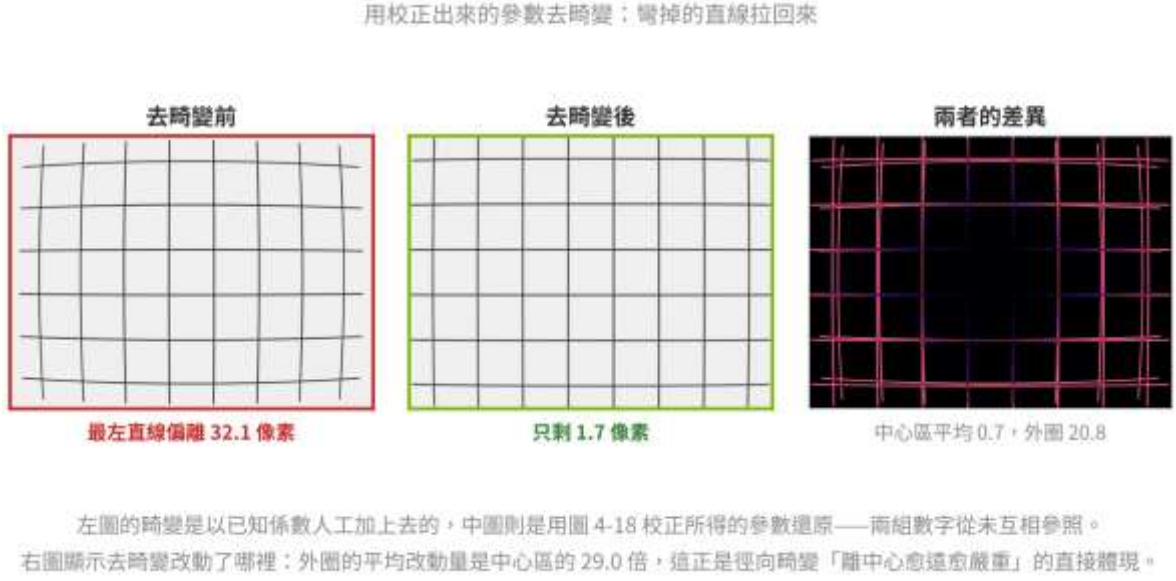


圖 4-19　用校正出來的參數去畸變

4.4.4 從像素到公尺

校正的最終價值，是讓相機成為量測工具。一旦知道內參，加上一個已知的參考(例如物體在已知距離的平面上、或畫面中有已知尺寸的參考物)，就能把「像素」換算成「公釐」。這個換算是後續多章的基礎：第 7.6.3 節由產線速度與像素尺寸推算運動模糊長度、第 12.7.2 節由車輛的像素位移換算實際車速、第 17 章機器人以視覺估計障礙物距離——它們全都建立在本節的校正之上。

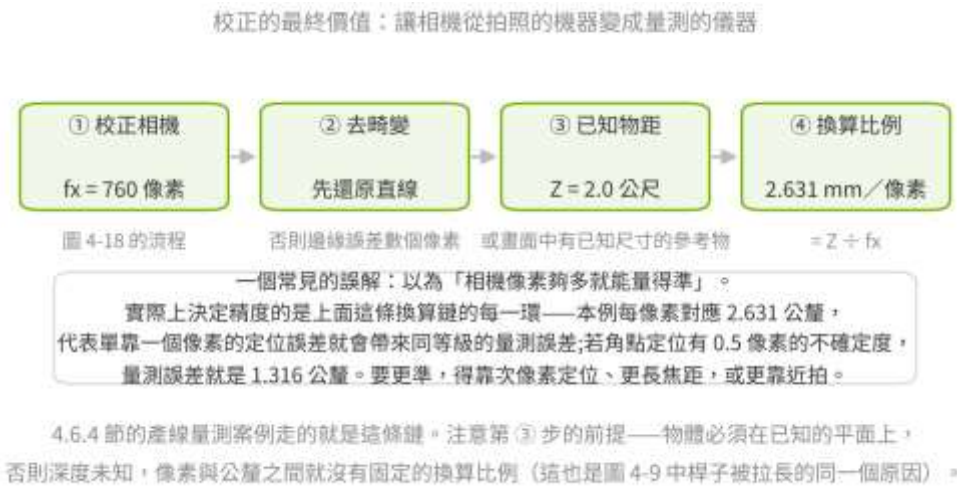


圖 4-21　從像素到公釐：讓相機變成量測儀器

4.5 幾何轉換在全書中的角色

幾何轉換是本書中出現頻率最高的工具之一，值得在此把散落各章的應用收攏起來：

- 資料增強(第 14、15、16 章)：訓練模型時的隨機旋轉、平移、翻轉、縮放，是對抗過度擬合最有效的手段之一，而它們全是本章的仿射轉換。第 16 章 YOLO 訓練參數中的 degrees、fliplr 正是如此。
- 視訊穩定(第 12 章)：估計相鄰影格之間的抖動(以第 6 章相位相關或第 13 章特徵匹配)，再用本章的幾何轉換反向補償，把每一影格「拉回」穩定位置。
- 影像拼接(第 13 章)：求出兩張照片間的透視轉換，把一張拉到與另一張對齊後融合，即成全景圖。
- 物件定位(第 13 章)：特徵匹配加 RANSAC 求出的單應性矩陣，能把參考影像的四個角投影到場景中，框出物件的位置與姿態。
- 車牌辨識(第 13 章)：斜拍的車牌先以透視轉換拉正，字元分割與辨識才能可靠進行。
- 鳥瞰圖與導航(本章 Lab、第 12 章)：把地面透視轉換成俯視圖，是循跡機器人與車道偵測的標準前處理。

- 機器人座標系(第 17 章):ROS 2 的 tf2 以齊次座標矩陣管理相機、機械臂、底盤、世界之間的座標轉換關係。

這份清單說明了一件事：本章不是一個「學完就過去」的獨立章節，而是往後每一章都會回頭取用的工具箱。建議你把程式 4-4 產出的相機參數檔妥善保存，它會陪你走完整個學期。

4.6 應用案例

4.6.1 案例一：文件掃描 App

情境：用手機斜拍一張紙本文件，自動偵測紙張邊界並拉正成正面的掃描檔。這是透視轉換最貼近生活的應用，完整流程串起本書多章：轉灰階與去雜訊(第 5 章)→ Canny 邊緣偵測(第 9 章)→ 找出最大的四邊形輪廓(第 9、10 章)→ 取四個角以透視轉換拉正(本章)→ 自適應門檻二值化成清晰的黑白掃描檔(第 9 章)。

工程細節值得留意：目標矩形的長寬比不能隨便設，否則文件會被不自然拉伸——程式 4-2 用「量測對邊長度取平均」的方式估計合理比例。若已知是 A4 紙，也可直接指定 1:1.414 的比例。這種「利用先驗知識提升結果」的思路，在工程上總是划算的。

4.6.2 案例二：機器人導航的鳥瞰圖

情境：循跡機器人或自駕車的前視相機，看到的地面是有透視變形的——遠處的車道線看起來會匯聚。若直接在原始影像上判斷「線在左邊還是右邊、彎曲程度多少」，會因透視而失真。

解法是逆透視映射(IPM)：選取地面上的一塊梯形區域，把它透視轉換成矩形，得到俯視的鳥瞰圖。在鳥瞰圖上，平行的車道線真的是平行的、彎道的曲率也能正確量測，後續的第 9 章霍夫轉換或曲線擬合都變得單純許多。這是車道保持與循跡機器人的標準前處理，也是本章 Lab 4-2 的目標。

前提條件必須說清楚：IPM 假設所有內容都在同一個平面(地面)上。因此地面上的車道線會被正確攤平，但立體的物件(其他車輛、行人)在鳥瞰圖中會被拉成怪異的長條——因為它們違反了平面假設。理解這個限制，你就知道鳥瞰圖適合用來分析路面標線，而不適合用來判斷障礙物的形狀。

4.6.3 案例三：資料增強的幾何操作

情境：訓練一個工件瑕疵偵測模型，但標註樣本只有兩百張。資料增強能以幾何轉換人工擴充資料的多樣性：隨機旋轉正負十五度、隨機平移百分之十、水平翻轉、隨機縮放。每一張原始影像因此能衍生出許多變化，逼迫模型學習「不管工件怎麼擺放都要認得」的穩健特徵。

但增強必須合理，這是最容易被忽略的一點：若你的工件永遠以固定方向進料，那麼旋轉一百八十度的增強反而是在教模型學習不會發生的情況，可能稀釋了有效的學習。同理，數字辨識任務不該做垂直翻轉(6 會變成 9 的樣子)、醫學影像不該做左右翻轉(器官有左右之分)。增強要模擬「真實世界可能出現的變化」，而非隨意變形。這個判斷力，是第 15 章訓練模型時的必修課。

4.6.4 案例四：相機校正在產線量測的價值

情境：以視覺量測工件尺寸，要求精度達到零點一毫米。此時鏡頭畸變絕不能忽略——未校正的廣角鏡頭在畫面邊緣可能有數個像素的位置偏差，換算成實際尺寸就是零點幾毫米的誤差，直接讓量測失效。

標準做法：出廠時以張氏標定法校正相機並存檔內參與畸變係數(程式 4-4)→ 每次取像後先去畸變 → 以已知尺寸的標準件或標定板建立「像素對實際尺寸」的換算比例 → 定期(如每季或更換鏡頭後)重新校正驗證。

這個案例呼應了叫獸開講的核心：在張氏標定法問世之前，要做到這件事需要昂貴的三維標定物，只有大企業玩得起;而現在，一張列印的棋盤格加上開源的 OpenCV，

任何一間中小企業、任何一個學生專題都能達成工業級的量測校正。降低門檻的力量，莫過於此。

4.7 本章與全書的觀念連結總表

本章觀念	後續應用的章節	如何延伸
齊次座標	第 17 章	ROS 2 的 tf2 座標框架管理
仿射轉換	第 12、14-16 章	視訊穩定的運動補償;訓練時的資料增強
透視轉換/單應性	第 13 章	RANSAC 求單應性做物件定位與影像拼接
內插與 INTER_AREA	第 3、9、16 章	取樣定理與防混疊;遮罩須用 NEAREST
相機校正與內參	第 7、12、17 章	推算運動模糊長度;像素換算車速;機器人測距
鏡頭畸變	第 9、13、17 章	直線偵測失準;特徵位置偏移;量測誤差
鳥瞰圖(IPM)	第 9、12 章	車道偵測與循跡機器人的前處理

表 4-2 第 4 章與全書其他章節的觀念連結

程式實作 Lab 4

Lab 4-1 轉換家族與內插實驗

目標：建立對轉換與內插的直覺。(a)以程式 4-1 對一張照片分別做平移、旋轉、縮放，並驗證「先旋轉再平移」與「先平移再旋轉」結果不同;(b)實作矩陣串接：比較「分三次做」與「合成一次做」的執行時間與 PSNR;(c)執行程式 4-3 的「連續旋轉十次轉回原位」實驗，記錄 PSNR 的劣化程度，說明為什麼理論上該回到原圖卻沒有;(d)對一張分割遮罩分別用 LINEAR 與 NEAREST 放大，用 np.unique 驗證前者產生了無

意義的中間值;(e)整理出一張「什麼情況用哪種內插」的決策表。

Lab 4-2 相機校正與機器人鳥瞰圖

目標：替課程相機建立專屬校正檔，並產生導航鳥瞰圖。(a)列印棋盤格(OpenCV 官網可下載)，固定於硬板上——務必平整，不可貼在會彎曲的軟紙板;(b)拍攝 15 張涵蓋畫面四角、遠近與傾斜各異的照片;(c)執行程式 4-4，記錄重投影誤差與內參，將 K 與 dist 存成 npz 檔供整學期所有實驗使用;(d)去畸變後，在地面貼四張定位貼紙圍成已知尺寸的矩形，以透視轉換產生俯視鳥瞰圖;(e)量化實驗：刻意只用 3 張照片重新校正，比較重投影誤差與 15 張時的差異，驗證叫獸開講思考題的結論;(f)ESP32-CAM 的廣角鏡畸變特別明顯，校正前後直線的彎曲程度差多少?請量化它。

Lab 4-3 文件掃描器

目標：做出手機掃描 App 的核心功能。(a)以程式 4-2 為基礎，用 cv2.setMouseCallback 讓使用者在影像上依序點四個角，取代寫死的座標;(b)依四點自動計算目標矩形的長寬比，避免文件被不自然拉伸;(c)串接第 3 章的成果——輸出前將文件轉灰階並提高對比，存成清晰的掃描檔;(d)挑戰題：第 9 章學會輪廓偵測後，回來把「手動點四角」升級成「自動找四角」，做成全自動掃描器;(e)延伸：實作反向轉換，把一張圖片貼到原始影像中那張紙的位置上——這正是擴增實境的基本原理。

Lab 4-4 資料增強管線(第 15 章前導)

目標：提前建立資料增強的觀念與工具。(a)寫一個 augment(img) 函式，隨機套用旋轉(正負 15 度)、平移(10%)、縮放(0.9 到 1.1 倍)、水平翻轉;(b)對一張影像產生 20 個增強版本並拼貼顯示，檢查是否有出現不合理的變形(如內容被轉出畫面外);(c)思辨題：針對三種不同任務——工件瑕疵偵測、手寫數字辨識、胸部 X 光判讀——各自判斷哪些增強該用、哪些不該用，並說明理由;(d)這個函式將在第 15、16 章訓練模型時直接使用，請存成共用模組。

Lab 4-5 內插法的代價實測

目標：把圖 4-5、4-12、4-14 的現象變成你自己量過的數據。(a)取一張清晰的照片，連續旋轉 36 度十次，每次都存檔;計算每一步與原圖的 PSNR 並畫成曲線，觀察劣化如何累積;(b)改成把十個矩陣相乘後只做一次 warpAffine，比較最終的 PSNR 與執行時間;(c)以 Laplacian 的標準差當作銳利度指標，量測兩種做法的差距——注意只能在兩者都有內容的中央區域比較，想一想為什麼不能直接比整張;(d)把同一張圖分別以 NEAREST、LINEAR、CUBIC、LANCZOS4 放大八倍，沿一條穿過強邊緣的水平線畫出灰階剖面，找出哪幾種會產生過衝，並解釋它與第 6 章振鈴的關係;(e)把一張圖縮小 1.5、2、4、8 倍，每個倍率都用 LINEAR 與 AREA 各做一次，量測混疊程度（提示：可用第 3 章的環帶圖當素材），找出「從幾倍開始非用 AREA 不可」;(f)把結論寫成三行口訣，貼在你的工具箱模組開頭。

Lab 4-6 校正你自己的相機

目標：為課程用的相機建立一份會陪你走完整學期的校正檔。(a)列印棋盤格並貼在硬質平板上——務必平整，貼在軟紙板上是重投影誤差偏大的頭號原因;(b)拍攝 15 到 20 張，刻意涵蓋多種傾角與畫面的各個區域（特別是四個角落，那裡的畸變最大、資訊最多）;(c)跑程式 4-4，記錄重投影誤差與內參，並把每張照片各自的誤差也印出來——找出誤差最大的那一兩張，看看它們有什麼共同點;(d)刻意做一組對照實驗：只用三張方位相似的照片校正一次，比較它的重投影誤差與用全部照片的結果。若前者的數字反而比較好看，請解釋為什麼（見習題 23）;(e)用校正參數對一張含直線的照片去畸變，量測某條直線在去畸變前後偏離直線的最大距離，以數字驗證效果;(f)拿一把尺或一個已知尺寸的物件，在固定距離下量測它的像素長度，建立「每像素幾公釐」的換算，再量另一個已知物件驗證誤差有多大;(g)把 camera_params.npz 存進你的工具箱倉庫。第 12 章的視訊穩定與第 17 章的部署都會直接讀它。

本章重點整理

- 幾何轉換改變像素位置而非數值;齊次座標以「升一維」把平移、旋轉、縮放統一為矩陣乘法，多重轉換可預先相乘合成單一矩陣，一次取樣完成(更快且品質更好);矩陣相乘順序不可交換，從右往左作用。
- 轉換家族層層包含：剛體(3 自由度，保長度角度)、相似(4)、仿射(6 自由度、3 組對應點、保平行)、透視(8 自由度、4 組對應點、保直線)。單應性成立的前提是拍攝平面或相機純旋轉。
- 反向映射避免破洞與衝突，但來源座標非整數故需內插;NEAREST 最快且不產生新值(遮罩專用)、LINEAR 為預設甜蜜點、CUBIC 高品質放大、AREA 為縮小首選(內建低通防混疊，呼應第 3 章)。
- 針孔模型的內部參數(fx、fy、cx、cy)屬於相機本身，外部參數(R、t)描述相機位姿;換鏡頭需重新校正內參，搬動相機只改外參。
- 徑向畸變分桶狀(係數負，廣角鏡)與枕狀(係數正，望遠端)，另有切向畸變;畸變破壞「直線成像仍為直線」的假設，使邊緣偵測、特徵匹配、視覺量測全部失準，故須先校正再量測。
- 張氏標定法(1998 MSR-TR-98-71,2000 TPAMI)只需印一張棋盤格、從至少兩個未知方位拍攝即可校正，為 OpenCV calibrateCamera 之核心;以重投影誤差驗收，一般小於 0.5 像素為佳。
- 幾何轉換是全書最常取用的工具：資料增強(第 14-16 章)、視訊穩定(第 12 章)、影像拼接與物件定位(第 13 章)、車牌拉正(第 13 章)、鳥瞰圖(第 9、12 章)、ROS 2 座標系(第 17 章)。

習題

1. 為什麼平移無法用 2×2 矩陣表示?齊次座標如何解決這個問題?統一之後帶來哪兩個好處?

   提示與參考解答：因為平移是「加法」而非線性的矩陣乘法，縮放與旋轉可寫成 2×2 矩陣乘法，平移不行。齊次座標替每個點多加一維(x, y, 1)，使平移也能寫成

3×3 矩陣乘法。兩個好處：一、多個轉換可預先相乘合成單一矩陣，一次做完;二、只需取樣內插一次，避免多次內插造成的品質劣化。

2. 寫出「先繞原點旋轉 90 度、再平移 (10, 20)」的合成 3×3 矩陣，並驗算點 (1, 0) 的落點。若順序顛倒，結果會相同嗎?

   提示與參考解答：旋轉 90 度矩陣 R = [[0,-1,0],[1,0,0],[0,0,1]];平移 T = [[1,0,10],[0,1,20],[0,0,1]]。合成 M = T·R(從右往左：先旋轉後平移)。點 (1,0,1) 經 R 得 (0,1,1)，再經 T 得 (10,21)。順序顛倒(M = R·T)結果不同：(1,0) 先平移得 (11,20)，再旋轉得 (-20,11)。矩陣乘法不可交換，順序必須明確。

3. 請比較剛體、相似、仿射、透視四種轉換的自由度、所需對應點數與保持的性質。

   提示與參考解答：剛體：3 自由度、2 組對應點、保長度與角度。相似：4 自由度、2 組點、保角度與形狀比例。仿射：6 自由度、3 組點、保平行性與面積比。透視：8 自由度、4 組點、保直線性與交比(不保平行，故能表現近大遠小)。可對照表 4-1。

4. 斜拍的棋盤，格線在影像中不再平行。應以仿射還是透視轉換拉正?為什麼?單應性成立的前提是什麼?

   提示與參考解答：應用透視轉換。因為仿射保持平行性，無法描述「平行線相交」的透視效果;透視轉換不保平行但保直線，正好對應近大遠小的投影。單應性成立的前提：拍攝的是一個平面(如紙張、地面、牆面)，或相機只做純旋轉而不平移。若場景有深度變化且相機平移，單一單應性無法描述兩影像的關係。

5. 為什麼幾何轉換採用反向映射而非正向映射?反向映射帶來什麼新問題?

   提示與參考解答：正向映射(把輸入像素搬到目標位置)會產生兩個問題：某些輸出位置沒有任何輸入落上去而形成破洞，某些位置則多個輸入重疊而衝突。反向映射對每個輸出像素反算來源，保證每個輸出都有值且唯一。新問題是：反算出的來源座標幾乎都不是整數，必須從鄰近整數像素估值——這就是內插，也是幾何轉換必然伴隨品質損失的原因。

6. 放大分割遮罩(數值只有 0 與 1)時，為什麼必須用最近鄰內插?用雙線性會發生什麼事?

提示與參考解答：因為最近鄰直接取最近像素的值，不會產生新數值，遮罩仍只有 0 與 1。雙線性是加權平均，會在邊界產生 0.3、0.5 這類中間值，而標籤圖的「0.5 類」毫無意義，後續處理會出錯。這也適用於第 9、16 章的分割結果與第 14 章的標籤資料。

7. 縮小影像為什麼建議用 INTER_AREA?這與第 3 章的哪個定理有關?

提示與參考解答：縮小就是降低取樣率，依第 3 章的 Nyquist 取樣定理，超過新取樣率一半的高頻成分會摺疊成混疊(假圖案)。INTER_AREA 以區域平均進行降取樣，本身內建了低通濾波的動作，先把高頻濾掉再取樣，因此能有效抑制混疊，是縮小的建議選項。

8. 說明內部參數與外部參數的差別。換一顆鏡頭後，哪一組需要重新校正?把相機搬到另一個腳架上呢?

提示與參考解答：內部參數(fx、fy、cx、cy)屬於相機與鏡頭本身，描述成像的幾何特性，與相機放在哪裡無關。外部參數(R、t)描述相機在世界中的位置與朝向。換鏡頭或調整變焦會改變成像特性，必須重新校正內參;搬到另一個腳架只改變位置與朝向，內參不變、只有外參改變(在機器人中由第 17 章的 tf2 即時追蹤)。

9. GoPro 影像中筆直的地平線變成弧形，這是哪種畸變?對應的係數正負為何?為什麼畸變對視覺量測是致命的?

提示與參考解答：這是桶狀畸變(barrel distortion)，徑向畸變係數為負，影像向外鼓、直線變成外凸弧線，常見於廣角鏡、GoPro、ESP32-CAM。致命原因：它破壞了「直線成像仍為直線」的假設，使第 9 章霍夫直線偵測失準、第 13 章特徵位置系統性偏移、第 17 章像素換算實際距離產生誤差，且愈靠畫面邊緣誤差愈大。故須先校正再量測。

10. 張氏標定法相較「精密三維標定物」方法的核心優勢是什麼?實務上為何要拍 10 張以上?

提示與參考解答：核心優勢：只需一張列印的平面棋盤格，從至少兩個不同方位觀察即可，且相機或棋盤皆可自由移動、運動不需已知;傳統方法需要精密加工的兩三個正交平面，昂貴且門檻高。張正友論文稱它「讓三維電腦視覺從實驗室往真實世界前進一步」。拍 10 張以上是為了對抗影像雜訊與角點定位誤差、並涵蓋畫面各角落以準確估計畸變;原論文實驗亦顯示估計的不確定度隨影像張數增加而下降。

11. 重投影誤差 1.8 像素算好嗎?請列出三個可能原因與對應的改善做法。

    提示與參考解答：不好，一般以低於 0.5 像素為佳，1.8 表示校正品質不佳。三個可能原因與對策：一、棋盤格不平整(貼在會彎曲的軟紙板上)——改貼在硬質平板;二、照片模糊或有動態模糊，角點偵測不準——用腳架、加強打光、縮短曝光;三、方位變化不足，所有照片角度相似——刻意涵蓋畫面四角、不同傾斜與遠近。另可檢查是否誤設棋盤格內角點數。

12. (綜合題)承 4.6.3 節：針對「工件瑕疵偵測」、「手寫數字辨識」、「胸部 X 光判讀」三種任務，請各自判斷旋轉 180 度與左右翻轉這兩種增強是否適用，並說明理由。

    提示與參考解答：工件瑕疵偵測：若工件以固定方向進料，旋轉 180 度是在教模型學習不會發生的情況，可能稀釋有效學習，不建議;若工件會任意擺放則適用。左右翻轉通常可行(瑕疵無左右之分)。手寫數字辨識：旋轉 180 度不適用(6 與 9 會混淆);左右翻轉不適用(鏡像數字不存在於真實資料)。胸部 X 光：旋轉 180 度不適用(解剖方向固定);左右翻轉不適用，因人體器官有左右之分(心臟偏左)，翻轉會產生醫學上錯誤的影像。原則：增強應模擬真實世界可能出現的變化。

13. 計算題：承圖 4-4。寫出「繞影像中心 (cx, cy) 旋轉 θ 角」的 3×3 齊次矩陣。提示：這不能只用一個旋轉矩陣。請說明為什麼要用三個矩陣相乘，以及它們的順序。

    提示與參考解答：單純的旋轉矩陣是繞原點（影像左上角）轉，要繞中心轉必須先把中心搬到原點、轉完再搬回去，因此答案是 $M = T(cx, cy) \cdot R(\theta) \cdot T(-cx, -cy)$。順序由右往左讀：先平移 $(-cx, -cy)$ 把中心移到原點，再旋轉 θ，最後平移 (cx, cy) 搬回原位。三者缺一不可，順序也不能調換——若寫成 $R \cdot T \cdot T^{-1}$ 就等於只做了旋轉。OpenCV 的 cv2.getRotationMatrix2D(center, angle, scale) 內部做的正是這件事，它直接回傳合成後的 2×3 矩陣。這個「搬到原點、做事、搬回去」的模式在圖形學中稱為共軛轉換，繞任意點縮放、沿任意軸鏡射都用同一招。

14. 計算題：承圖 4-5。對一張影像連續做十次「旋轉 36 度」，理論上會回到原位。請說明為什麼實際結果會明顯劣化，以及正確的做法。若改成連續做十次「旋轉 36 度再轉回 −36 度」呢?

提示與參考解答：因為每一次 warpAffine 都要做一次內插，而內插本質上是加權平均，等同於一次輕微的低通濾波（第 5 章）。十次內插就是十次低通，高頻細節被逐次抹平，影像愈來愈糊，而且邊界每轉一次就被裁掉一點。正確做法是先把十個矩陣相乘得到單一矩陣（本例乘起來恰好是單位矩陣），最後只呼叫一次 warpAffine——如果合成結果真的是單位矩陣，甚至連呼叫都不必。至於「轉 36 度再轉回 –36 度」重複十次：劣化會更嚴重，因為這樣做了二十次內插。一個常見的誤解是「反向轉換可以抵銷正向轉換」——幾何上可以，資訊上不行：第一次內插丟掉的高頻，第二次內插不會還給你。這與第 3 章「取樣與量化不可逆」是同一個道理。

15. 承圖 4-6 與圖 4-7：為什麼仿射需要 3 組對應點、透視需要 4 組?請由自由度推算。若你手上有 10 組對應點，多出來的那些有什麼用?該如何使用?

提示與參考解答：每一組對應點提供兩個方程式（x 與 y 各一）。仿射有 6 個自由度，6 ÷ 2 = 3 組;透視的 3×3 矩陣有 9 個元素，但整體乘上一個常數不改變結果（齊次座標的尺度不計），所以實際自由度是 8，8 ÷ 2 = 4 組。多出來的對應點有兩個用途。其一是降低雜訊影響：點的座標本身有定位誤差，用最小平方法在所有點上求一個「整體最吻合」的解，比只用最少點數穩定得多。其二是偵測並排除錯誤：若其中有幾組是錯誤匹配（outlier），最小平方會被它們嚴重拉偏，此時要改用 RANSAC（第 13 章）——隨機取最少點數求解、統計有多少點同意這個解、重複多次取最多人同意的那個。OpenCV 的 cv2.findHomography(src, dst, cv2.RANSAC) 正是這個流程。

16. 承圖 4-9：單應性成立的兩個前提是什麼?請各舉一個「前提成立」與「前提不成立」的實例，並說明前提不成立時會看到什麼現象。

提示與參考解答：兩個前提，滿足其一即可。其一，被拍攝的內容全部位於同一個平面上（紙張、地面、牆面、棋盤格）。其二，相機只做純旋轉而不平移（原地轉動拍全景，此時深度差異不產生視差）。前提成立的實例：手機掃描 App 拉正桌上的文件、循跡機器人把地面轉成鳥瞰圖、原地旋轉拍攝的全景接圖。前提不成立的實例：在街上邊走邊拍大樓做全景——相機平移了，而場景有明顯深度（大樓、路樹、遠山各在不同距離）。現象是：你可以讓其中一個深度層對齊得很好，但其他深度層會出現明顯的錯位與鬼影，而且怎麼調參數都無法同時對齊。這不是演算法不夠好，是單一個 8 自由度的矩陣在數學上就描述不了「不同深度有不同位移」這件事——那需要第 12 章的光流或完整的三維重建。

17. 承圖 4-10：為什麼幾何轉換要用反向映射而非正向映射?正向映射的兩個問題各自在什麼情況下最嚴重?反向映射完全沒有缺點嗎?

    提示與參考解答：正向映射有兩個問題。破洞：放大時最嚴重——輸出像素比輸入多，必然有些輸出位置沒有任何輸入落上去。衝突：縮小時最嚴重——多個輸入擠到同一個輸出位置，後寫的覆蓋先寫的，等於隨機取樣（而非平均），會產生第 3 章的混疊。反向映射對每個輸出像素反算來源，保證每個輸出都有值、也不會重複寫入，兩個問題一次解決。但它不是沒有代價：其一，需要轉換矩陣可逆（所幸本章的轉換都可逆）;其二，反算出的來源座標不是整數，必須內插，而內插本身會損失高頻（圖 4-5 與圖 4-12）;其三，縮小時反向映射只取樣一個點，同樣會混疊——這正是縮小要特別用 INTER_AREA 的原因（圖 4-14）。換句話說，反向映射解決的是幾何上的完整性，取樣理論上的問題仍然要另外處理。

18. 計算題：一個像素的來源座標被反算為 (217.3, 154.8)。已知四個鄰近像素的值為 I(217,154)=100、I(218,154)=140、I(217,155)=160、I(218,155)=200。請以雙線性內插算出該像素的值，並說明計算的順序。

    提示與參考解答：令小數部分 a = 0.3（x 方向）、b = 0.8（y 方向）。先在上面那一列做一維線性內插：100 × (1 − 0.3) + 140 × 0.3 = 70 + 42 = 112。再在下面那一列做同樣的事：160 × 0.7 + 200 × 0.3 = 112 + 60 = 172。最後在兩個結果之間依 b 內插：112 × (1 − 0.8) + 172 × 0.8 = 22.4 + 137.6 = 160.0。所以該像素的值為 160。順序可以顛倒（先直後橫）而結果相同，這稱為可分離性，與第 5 章可分離濾波是同一個性質，也是雙線性內插快的原因。補充一個檢查習慣：四個權重 (1−a)(1−b)、a(1−b)、(1−a)b、ab 相加必為 1，算完可以驗算一下。

19. 承圖 4-13：放大一張只有 0 與 2 兩種值的標籤圖，用雙線性內插後會發生什麼事?為什麼這在語意分割中是嚴重錯誤?那麼「把雙線性的結果四捨五入回整數」可行嗎?

    提示與參考解答：雙線性是加權平均，在 0 與 2 的交界處會產生 0.4、1.0、1.7 這類中間值，取整後出現「1」——而標籤 1 可能代表完全不同的類別（例如 0 是背景、2 是車輛，1 卻是行人）。於是模型憑空在每個物件的邊緣繞上一圈「行人」，這在語意分割的評分（第 16 章的 IoU）中是直接的錯誤。至於四捨五入：不可行，因為標籤值是「名目尺度」而非「數值尺度」——類別 0、1、2 之間沒有大小或距

離關係，在它們之間做平均再取整，本身就沒有意義。正確做法只有一個：標籤圖一律用 INTER_NEAREST。同樣的原則適用於第 10 章的二值遮罩、第 9 章的分割結果，以及任何「數值代表身分而非數量」的影像。

20. 承圖 4-14：縮小影像時 INTER_AREA 與 INTER_LINEAR 的差別是什麼?為什麼 INTER_LINEAR 在縮小時會失效?請用第 3 章的取樣定理解釋，並說明縮小 1.5 倍與縮小 8 倍哪一種情況差別更大。

    提示與參考解答：INTER_LINEAR 不論縮放倍率，永遠只看來源座標周圍的 2×2 個像素。縮小 8 倍時，輸出的每一個像素其實代表輸入的 8×8 = 64 個像素，但雙線性只看了其中 4 個，其餘 60 個像素的資訊被直接忽略——這等同於第 3 章的「直接跳點取樣」，必然混疊。INTER_AREA 則會依縮放倍率自動把對應區域內的所有像素平均，等於內建了一個隨倍率調整的低通濾波器，先濾掉留不住的高頻再取樣，符合取樣定理。倍率愈大差別愈明顯：縮小 1.5 倍時，每個輸出像素對應 2.25 個輸入像素，雙線性看的 4 個已涵蓋大部分，差別不大;縮小 8 倍時只看了 64 分之 4，差別極大。實務上的判準是：縮小超過 2 倍就該用 AREA，或先用高斯模糊再縮。

21. 承圖 4-16：內部參數 fx、fy、cx、cy 各自的幾何意義是什麼?若把相機的解析度由 1920×1080 改成 960×540（二分之一取樣），這四個數字該怎麼換算?

    提示與參考解答：fx 與 fy 是以像素為單位的焦距，決定「同樣的物體在影像中有多大」;兩者相等代表感光元件的像素是正方形，不相等則影像在某個方向被拉長。cx、cy 是主點，也就是光軸與成像面的交點，理想上落在影像中心，實際上因組裝公差而略有偏移。解析度減半時，四個數字全部乘以 0.5：fx、fy 減半是因為同一個視角現在對應一半的像素數;cx、cy 減半是因為影像中心的像素座標也減半了。畸變係數則不變，因為它們定義在正規化座標上，與解析度無關。這是實務上極常見的陷阱：用高解析度校正、卻在低解析度上推論，忘了換算內參，結果整條管線的量測全錯而且錯得很一致，不容易發現。

22. 承圖 4-17 與 4-19：為什麼徑向畸變在畫面中心幾乎不影響、愈往邊緣愈嚴重?請由畸變模型的數學形式說明。這對「只用畫面中央區域」的設計策略有什麼啟示?

    提示與參考解答：徑向畸變的模型是把正規化座標乘上一個只與半徑有關的倍率：$(1 + k1{\cdot}r^2 + k2{\cdot}r^4 + k3{\cdot}r^6)$，其中 r 是該點到主點的距離。在中心 r = 0，倍率恰為

1，位置完全不變;離中心愈遠 r 愈大，而且因為是 r 的平方、四次方、六次方，偏移量增長得非常快——這就是為什麼畫面四角最誇張。圖 4-19 實測的數字正是這個形狀：去畸變在中心區的平均改動量接近零，外圈卻大上數十倍。啟示是：若應用允許，把關鍵的量測區域安排在畫面中央，可以大幅降低對校正精度的依賴。這在產線設計上很實用——寧可換一顆視角較窄的鏡頭並把工件放在中央，也不要用廣角鏡硬塞進整個視野再靠軟體救。但要注意這是降低風險而非免除校正：要達到零點一毫米等級的量測，中央區域仍然需要去畸變。

23. 承圖 4-18 與 4-20：重投影誤差是怎麼算出來的?為什麼它能作為校正品質的驗收標準?若某次校正的重投影誤差是 0.08 像素，這一定是好事嗎?

    提示與參考解答：計算方式是把求得的內參、畸變係數與每張照片的外參，拿去把棋盤格角點的「世界座標」重新投影回影像，再與當初偵測到的角點位置相比，取所有點的均方根距離。它之所以能當驗收標準，是因為它直接衡量「這組參數能不能解釋觀測到的資料」——參數愈接近真實，投影回去就愈吻合。但 0.08 像素不一定是好事，這題的重點在這裡。重投影誤差只衡量「在這批照片上吻合得多好」，並不保證參數本身正確。兩種常見的假好成績：其一，照片太少或方位太相似，模型有足夠自由度去遷就這幾張，等於過度擬合;其二，誤差被平均稀釋——若多數角點很準、少數幾張照片很糟，平均起來仍然好看。正確的驗收應該包含三件事：看每張照片各自的誤差而非只看總平均、確認拍攝方位涵蓋了畫面各區域與多種傾角、以及最重要的——拿一個已知尺寸的標準件實際量一次。這與第 3.5 節「指標是工具，不是目的」完全同源。

24. 承圖 4-21 與 4.6.4 節：要以視覺量測工件尺寸達到 0.1 毫米的精度，為什麼一定要先做相機校正?請估算：若未校正的廣角鏡在畫面邊緣有 5 像素的位置偏差，而每像素對應 0.05 毫米，這會造成多大的量測誤差?這個誤差有什麼特性讓它特別危險?

    提示與參考解答：5 像素 × 0.05 毫米／像素 = 0.25 毫米，已經是要求精度 0.1 毫米的兩倍半，完全不可接受。若量測的是跨越畫面的長度，兩端各有偏差，誤差還會加倍。這個誤差有兩個危險的特性。其一，它是系統性的而非隨機的——多量幾次取平均完全沒有用，因為每次測量都朝同一個方向偏同樣的量。其二，它隨位置而變：同一個工件放在畫面中央量到的尺寸，與放在角落量到的不一樣。於是你會看到「同一批產品的量測值隨著它在輸送帶上的位置而漂移」這種極難除錯的現象—

> *—工程師通常會先去懷疑光源、機構、甚至產品本身，而不會想到是鏡頭。正確流程是：出廠時校正一次並存檔、每次取像後先去畸變、再以已知尺寸的標準件建立像素與公釐的換算比例，並定期用標準件複驗。*

25. (綜合設計題)你要為一台循跡機器人建立鳥瞰圖（逆透視映射）。相機裝在車體前方，俯角約 30 度，距地面 20 公分。請說明：(a) 你需要哪些資訊才能求出那個透視矩陣?有哪兩種取得方式?(b) 鳥瞰圖中「每像素代表幾公分」該怎麼決定?(c) 這套方法在哪些情況下會失效?(d) 車子行進中震動造成俯角改變 2 度，會有什麼後果?該怎麼辦?

> *提示與參考解答：本題無標準答案，評分重點在前提意識與失效分析。(a) 需要「地面上四個點在影像中的座標」與「這四個點在地面上的實際相對位置」。兩種取得方式：一是手動標定——在地面上貼四個標記、量好它們的實際間距，拍一張照後在影像上點出四個角，用 getPerspectiveTransform 求解，簡單可靠，適合相機固定的場合;二是由幾何推算——已知相機高度、俯角與內參，可直接計算地平面與影像平面之間的單應性，不必貼標記，但對安裝角度的精度要求高。實務上建議先用方法一做出可用版本，再用方法二做自動化。(b) 由你要偵測的最小特徵決定（第 3.7.3 節的思路）：若車道線寬 2 公分、希望有 4 個像素涵蓋，則每像素 0.5 公分;再由機器人的前視距離需求（例如要看到前方 1 公尺）決定鳥瞰圖的高度為 200 像素。(c) 至少四種失效情況：地面不平（上下坡、減速丘）使平面假設破裂;立體物件（其他機器人、人的腳）會被拉成長條，因為它們不在地面上;遠處區域在原影像中只佔少數像素，攤平後被嚴重放大而模糊，可用距離有上限;以及光照造成的陰影在鳥瞰圖中與車道線難以分辨。(d) 俯角改變 2 度會讓整張鳥瞰圖產生系統性的梯形偏移，遠處的誤差遠大於近處，表現為「車道線在鳥瞰圖中突然變寬或變斜」，導致控制器誤判而產生震盪。三種對策：機構上加裝減震並降低重心;軟體上對連續數影格的轉換矩陣做時間平滑;或加裝慣性感測器即時估計俯角並動態修正矩陣（這已是第 17 章感測器融合的範疇）。*

參考資料與延伸閱讀

- Zhang, Z., A Flexible New Technique for Camera Calibration, IEEE TPAMI, 22(11), 1330-1334, 2000.(技術報告版 MSR-TR-98-71, 1998 年 12 月;本章叫獸開講之史料來源)

- Zhang, Z., Flexible Camera Calibration by Viewing a Plane from Unknown Orientations, ICCV, 1999.
- Gonzalez, R. C., & Woods, R. E., Digital Image Processing, 4th ed., Pearson, 2018, Ch. 2.4(幾何轉換與內插)。
- Szeliski, R., Computer Vision: Algorithms and Applications, 2nd ed., Springer, 2022, Ch. 2、11(投影幾何與立體視覺)。
- Hartley, R., & Zisserman, A., Multiple View Geometry in Computer Vision, 2nd ed., Cambridge, 2004.(進階必讀)
- OpenCV 官方教學：Camera Calibration(含可下載之棋盤格圖檔)。
- 補充教材與課程資訊：@prof(https://page.line.me/prof)